Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Evaluating Tool-Using Language Agents: Judge Reliability, Propagation Cascades, and Runtime Mitigation in AgentProp-Bench

BHASKAR GURRAM[1]
[1]Zasti Inc., Ashburn, VA 20147 USA (e-mail: gurrambhaskar.ai@gmail.com)
Corresponding author: Bhaskar Gurram (e-mail: gurrambhaskar.ai@gmail.com).

ABSTRACT Automated evaluation of tool-using large language model (LLM) agents is widely assumed to be reliable, yet this assumption has rarely been validated against human annotation. We present AgentProp-Bench, a benchmark of 2,000 tasks across four domains with a 100-label human-validated subset, and use it to study three questions: judge reliability, error propagation, and runtime mitigation. We evaluate nine proprietary LLMs and four open-weight models (Qwen2.5 3B/7B/14B and Hermes-3-Llama-3.1-8B) served locally through an identical prompted tool-use harness. Substring-based judging agrees with human annotation at kappa = 0.049 (chance level), whereas a three-LLM ensemble judge reaches kappa = 0.432 (moderate) with a conservative bias. Under validated evaluation, a parameter-level injection propagates to a wrong final answer with a human-calibrated probability of about 0.62 (per-model range 0.46 to 0.73); the open-weight models behave identically (injection-to-error 0.63 to 0.74). Rejection (catching a bad parameter) and recovery (correcting after acceptance) are statistically independent (Spearman rho = -0.007). We also find that models vary by an order of magnitude in how often they emit a tool call (5 to 100 percent), and that some fabricate tool use, asserting a tool result they never obtained (Gemini-2.0-Flash in 42.5 percent of traces). A tuned runtime interceptor cuts GPT-4o-mini hallucination by 23.0 percentage points under a concurrent n = 600 control, but not for Gemini-2.0-Flash, which rarely emits an injectable tool call. All code, data, traces, and human labels are released.

INDEX TERMS Artificial intelligence, benchmark testing, error analysis, evaluation, large language models, natural language processing, software agents, tool-using agents.

I. INTRODUCTION

Tool-using large language model (LLM) agents translate natural-language queries into structured API calls, execute those calls, and synthesize the results into a final answer. When such an agent hallucinates, the failure is not confined to the surface token distribution: a mistyped parameter, an ignored tool error, or an overconfident summary can each corrupt the output. Agent benchmarks such as τ-bench [**?**], AgentBench [**?**], and BFCL [**?**] report a single end-to-end correctness number and cannot localize *where* a failure occurs or *how* it propagates through the pipeline.

A less visible but equally important gap concerns *how correctness is measured*. Most agent benchmarks rely on substring or keyword-match heuristics to judge whether an agent's final answer matches a reference. This practice has been questioned in the information retrieval (IR) community, where studies show that LLM judges and heuristic judges can disagree substantially on relevance assessments [**?**], [**?**], [**?**]. If the evaluation itself is unreliable, then every downstream claim about model ranking, error propagation, or mitigation effectiveness is suspect.

We present AgentProp-Bench, a benchmark and methodology designed to address both gaps. Our contributions are:

1) **Judge-reliability validation.** We show that substring-based evaluation of agent outputs agrees with human annotation at κ=0.049 (chance-level), while a three-LLM ensemble judge reaches κ=0.432 (moderate agreement), validated against 100 stratified human labels (Section **??**).
2) **Stage-level propagation measurement.** Under validated evaluation, we define three independent stage indicators and show that a parameter-level injection propagates to a wrong answer with human-calibrated probability $\approx$0.62 (range 0.46–0.73 per model) (Sec-

tion **??**).

3) **Rejection–recovery decomposition.** Rejection rate (catching bad parameters) and recovery rate (producing a correct answer despite accepting bad parameters) are statistically independent across thirteen models (Spearman ρ=0.041), revealing that models vary on two orthogonal axes of robustness (Section **??**).
4) **Tool-call adherence and fabricated tool use.** Models differ by more than an order of magnitude in how often they emit a tool call (5–100%), and some fabricate tool use—asserting a tool result they never obtained (Gemini-2.0-Flash: 42.5% of traces). A low stage-one rate can therefore reflect tool-call avoidance rather than robustness (Section **??**).
5) **Generalization to open-weight models.** Four open-weight models across two families (Qwen2.5 3B/7B/14B and Hermes-3-Llama-3.1-8B), served locally under the identical harness, show the same substantial propagation (injection-to-error 0.63–0.74), confirming the effect is not specific to proprietary APIs (Section **??**).
6) **Runtime interceptor under concurrent control.** A tuned three-layer Interceptor reduces hallucination on GPT-4o-mini by 23.0 pp under a concurrent n=600 no-interceptor control ($p < 0.001$), but has no significant effect on Gemini-2.0-Flash, which rarely emits an injectable tool call (Section **??**).
7) **Open benchmark release.** All 2,000 tasks, 14,750 agent traces, 100 human labels, analysis scripts, and agent source code are publicly released.

The remainder of this paper is organized as follows. Section **??** reviews related work on agent benchmarks, LLM-as-judge reliability, and agent hallucination. Section **??** states the research objectives. Section **??** describes the AgentProp-Bench dataset. Section **??** presents the methodology. Section **??** reports results. Section **??** discusses theoretical and practical implications. Section **??** acknowledges limitations. Section **??** concludes.

II. RELATED WORK

A. TOOL-USING LLM AGENT BENCHMARKS

The evaluation of tool-using LLM agents has rapidly expanded. τ-bench [**?**] evaluates agents on multi-step retail and airline tasks, measuring end-to-end task completion. [**?**] extend τ-bench to additional domains. AgentBench [**?**] spans eight environments including web browsing and database queries. BFCL [**?**] focuses on function-calling accuracy across multiple programming languages and API styles. MCPBench [**?**] evaluates agents on real-world API integration tasks. Recent surveys [**?**], [**?**] provide comprehensive overviews of agent architectures and evaluation practices.

These benchmarks share a common limitation: they report a single end-to-end correctness number and do not localize *where* in the agent pipeline a failure originates. AgentProp-Bench addresses this by decomposing evaluation into three independently measured stages.

B. LLM-AS-JUDGE RELIABILITY

Using LLMs as judges has become widespread in both NLP and IR evaluation. [**?**] introduced the LLM-as-judge paradigm with MT-Bench and Chatbot Arena. [**?**] survey the growing LLM-as-judge literature, identifying position bias, verbosity bias, and self-enhancement bias. [**?**] provide a multidimensional analysis of when LLM judges agree with humans.

In the IR community, judge reliability has received particular scrutiny. [**?**] argue that LLM judges should not replace human assessors without validation. [**?**] show that LLM judges and human searchers diverge on relevance criteria. [**?**] demonstrate that LLM judges can be fooled by surface-level text manipulation. [**?**] benchmark judge reliability across retrieval tasks. [**?**] extend judge evaluation to multilingual settings. [**?**] quantify when and why practitioners should trust LLM-based evaluation.

Our work contributes to this literature by validating an LLM-ensemble judge against 100 human labels in the specific context of tool-using agent evaluation, where output format variability (structured data, error messages, multi-step reasoning) makes heuristic evaluation particularly unreliable.

C. HALLUCINATION IN LLM AGENTS

Hallucination in standalone LLMs has been extensively surveyed [**?**], [**?**]. Agent-specific hallucination introduces additional failure modes: parameter fabrication, tool-output misinterpretation, and cross-turn memory corruption [**?**].

Most closely related to our work, [**?**] introduce AgentHallu, a benchmark for automated hallucination attribution in LLM-based agents. AgentHallu performs step-level attribution on 693 trajectories across five domains, categorizing hallucinations by stage of origin. AgentProp-Bench differs in three respects: (1) we use *controlled injection* rather than naturally occurring errors, enabling causal measurement of propagation rates; (2) we validate our evaluation methodology against 100 human labels (κ=0.432), whereas AgentHallu relies on automated attribution; (3) we evaluate a runtime interceptor under a concurrent control, providing mitigation evidence beyond diagnosis. The two benchmarks address complementary questions: AgentHallu asks "where did this hallucination come from?" while AgentProp-Bench asks "if we inject an error here, with what probability does it propagate there?"

D. RUNTIME MONITORING AND MITIGATION

Several approaches have been proposed for runtime hallucination detection and mitigation. Self-consistency voting [**?**] samples multiple reasoning paths and selects the majority answer. [**?**] propose SelfCheckGPT for detecting hallucinations without external references. [**?**] use semantic entropy to detect confabulations in free-form LLM generations. Guardrails and runtime enforcement frameworks [**?**] provide schema-level validation.

Our Interceptor combines schema validation, reasoning-keyword monitoring, and output-consistency checking in a three-layer architecture. Unlike self-consistency voting (which requires $k\times$ sampling cost), the Interceptor runs in

parallel with a single agent execution at negligible additional cost.

E. RELEVANCE TO IR AND IS EVALUATION

The evaluation-reliability problem we address has direct implications for the IR and information science (IS) communities. LLMs are increasingly used for relevance assessment in IR evaluation [**?**], and the reliability of these assessments directly affects system rankings. [**?**] study LLMs as relevance judges in retrieval settings. [**?**] examine synthetic data generation for IR evaluation. [**?**] survey LLM applications in information science.

Our finding that substring-based evaluation agrees with humans at κ=0.049 (chance-level) has implications beyond agent benchmarks. Substring and keyword-match heuristics are common correctness proxies in question-answering and retrieval-augmented generation (RAG) evaluation [**?**], [**?**]. The ensemble-judge methodology we validate here—with explicit per-rater and human-calibrated agreement reporting—transfers directly to these settings.

F. RESEARCH GAP

Existing work provides either end-to-end benchmarks without stage-level decomposition, or judge-reliability studies without application to agent evaluation. No prior work validates an agent evaluation pipeline end-to-end against human annotation, measures stage-conditional propagation under validated judging, and evaluates a mitigation under concurrent control—all in one study. AgentProp-Bench fills this gap.

III. RESEARCH OBJECTIVES

We formulate three research objectives:

a: RO1: Judge reliability.

What is the agreement between heuristic substring matching, a three-LLM ensemble judge, and human annotation on tool-using agent outputs? Prior work shows that heuristic evaluation is unreliable for paraphrased or verbose outputs [**?**]. We quantify this unreliability in the agent-evaluation context and validate the ensemble as a better—but imperfect—alternative.

b: RO2: Error propagation under validated evaluation.

How does a parameter-level injection propagate through the agent pipeline when correctness is measured by a validated judge? Prior work either reports end-to-end correctness or attributes errors post hoc [**?**]. We measure stage-conditional propagation rates and decompose model robustness into independent rejection and recovery capabilities.

c: RO3: Runtime mitigation under concurrent control.

Can a lightweight runtime interceptor reduce hallucination under a properly controlled experiment? Prior interceptor evaluations [**?**] typically compare against historical baselines. We evaluate under a concurrent n=600 no-interceptor control on the same task pool.

TABLE 1. Dataset composition of AgentProp-Bench. The benchmark comprises 2000 core tasks across four domains plus 300 held-out retail tasks. Difficulty was assigned during task creation based on required tool-chain length and reasoning depth. N = 2,000 core + 300 retail.

Domain	Tasks	Easy	Medium	Hard	Example query (truncated)
Calendar	667	192	282	193	`Create a '1:1 with`
Weather	674	215	248	211	`What is the weathe`
Medical	559	106	237	216	`Schedule a checkup`
Knowledge	100	0	46	54	`Use the search_kno`
Retail (held-out)	300	—	—	—	`Search product cat`

IV. DATASET: AgentProp-Bench

A. TASK DESIGN AND CONSTRUCTION

Each task in AgentProp-Bench is a tuple $(q, D, T_q, a^\star)$ where q is a natural-language query, D is a domain, T_q is a set of available tools, and $a^\star$ is a reference answer. Tasks are classified by difficulty: *easy* (single tool call), *medium* (two-tool chain), and *hard* (three-tool chain or branching logic). Tasks and reference answers were generated synthetically using domain-specific API simulators, then manually verified for correctness.

B. DOMAIN BREAKDOWN

AgentProp-Bench comprises 2,000 tasks across four domains plus a held-out retail domain:

- **Calendar** (667 tasks): event creation, schedule queries, conflict detection.
- **Weather** (674 tasks): current conditions, forecasts, historical comparisons.
- **Medical** (559 tasks): patient lookup, appointment scheduling, condition retrieval.
- **Knowledge** (100 tasks): general question-answering (ablation only, no tool injection).
- **Retail** (300 tasks, held-out): product search, detail inspection, order placement.

Table **??** provides a detailed breakdown by domain and difficulty.

C. TOOL SCHEMA AND SIMULATED ENVIRONMENT

Each domain provides 3–5 tools implemented as deterministic Python functions with JSON schemas. Tools validate input parameters against their schemas and return structured JSON responses. Invalid parameters produce explicit error messages rather than silent failures. This design ensures that parameter-level injection effects are observable in tool responses.

D. ERROR-INJECTION PROTOCOL

We use the P2 *semantic-wrong* injection protocol: before the agent's first tool call executes, we replace a semantically critical parameter with an incorrect but schema-valid value. For example, in a weather query for "London," we might replace the city parameter with "Manchester"—a valid city but the wrong one. The injection is applied at the `parameter_generation` stage of the pipeline.

E. TRACE COLLECTION

We collected agent traces across nine proprietary LLMs—GPT-4o, GPT-4o-mini, GPT-4.1-mini, GPT-4.1-nano, GPT-3.5-turbo, o3-mini (OpenAI); Gemini-2.0-Flash, Gemini-2.5-Flash (Google); and DeepSeek-V3 (via OpenRouter)—and four open-weight models spanning two families (Qwen2.5 3B/7B/14B and Hermes-3-Llama-3.1-8B) served locally with vLLM through the identical prompted-ReAct harness. For the P2 semantic-wrong condition, we evaluated 200 unique tasks per model for the six main proprietary models and all four open-weight models (800 rows each across four error types) and 50 tasks for three frontier proprietary models. The propagation analysis therefore covers 2,150 semantic-wrong traces across thirteen models (1,350 proprietary, 800 open-weight); the no-injection baseline covers 4,550 traces. Including all four injection types and the interceptor experiment, the released benchmark comprises over 14,000 agent traces. Proprietary models were accessed via their official APIs and open-weight models via a local vLLM server, all with temperature 0.

F. HUMAN LABEL COLLECTION

To validate the ensemble judge, we collected 100 human labels on a stratified sample of P2 semantic-wrong traces. The sample was stratified across all nine proprietary models (10–12 per model) and three domains (calendar, medical, weather). One annotator (the first author) labeled each trace as "correct" (YES) or "incorrect" (NO) by comparing the agent's final answer to the reference answer. The labeling criterion matched the ensemble judge prompt: accept minor wording differences and extra explanation as long as the core fact is correct. A second annotator (acknowledged) independently labeled all 100 traces using the same criterion, blind to the first annotator's labels. Inter-annotator agreement is Cohen's κ=0.835 (92% raw agreement), indicating almost-perfect agreement (Landis–Koch [**?**]) and establishing the human labels as a reliable reference for the judge comparison.

G. DATA AVAILABILITY

All data are publicly available at https://github.com/bhaskargurram-ai/agenthallu-bench. The repository includes task specifications, all result CSVs, analysis scripts, agent source code, and human annotation labels. Every numerical claim in this paper traces to a specific file via inline comments in the LaTeX source.

V. METHODOLOGY

A. JUDGE PROTOCOLS

We evaluate three judging methods:

a: Heuristic (substring match).

The agent's final answer is marked correct if the first 20 characters of the reference answer appear in the first 200 characters of the agent's answer, or if any word longer than 3 characters from the reference appears in the agent's answer. This mirrors common practice in agent benchmarks.

b: Three-LLM ensemble.

Three LLMs (GPT-4o, Gemini-2.5-Flash, GPT-4o-mini) independently judge each trace using an identical prompt: "Respond YES if the agent answer substantively matches the reference, NO otherwise. Accept minor wording differences as long as the core fact is correct." The majority vote (2/3 or 3/3 YES) is the ensemble verdict.

c: Human annotation.

One hundred traces were labeled by a human annotator using the same criterion as the ensemble prompt.

B. STAGE-LEVEL PROPAGATION METRIC

We define three stage indicators, computed **independently** from separate data columns:

- $S_1 = \mathbf{1}[\text{eps} \geq 1]$: the injection was applied (the agent's planned tool call was modified).
- $S_2 = \mathbf{1}[\text{eps} \geq 2]$: the modified call executed with an observable effect (parameter validation errors or altered tool response).
- $S_3 = \mathbf{1}[\texttt{ensemble_correct} = \texttt{False}]$: the final answer is wrong under ensemble judging.

S_1 and S_2 are derived solely from the `eps` column (computed by the EPS scorer during trace collection). S_3 is derived solely from the `ensemble_correct` column (computed by the post-hoc ensemble judge). No indicator references any other indicator's column.

a: Transparency note.

An earlier version of this analysis (submitted in the original manuscript) used a `classify_stage()` function that mapped `reached_output=True` to stage 3 and then defined S_2 as stage ≥ 2, which included those same rows. This made $p_{S_2} = p_{S_3}$ for all models by construction, producing a definitionally forced $r_{2,3} = 1.00$. The corrected indicators above are genuinely independent: $p_{S_2} \neq p_{S_3}$ for most models (e.g., DeepSeek-V3: $p_{S_2} = 7.0\%$ vs. $p_{S_3} = 62.5\%$).

We report the stage-reach probability $p_k = \mathbb{P}[S_k = 1]$, the hop-conditional rate $r_{k,k+1} = \mathbb{P}[S_{k+1} = 1 \mid S_k = 1]$ (computed on the $S_k = 1$ subset), and the injection-to-error rate $\mathbb{P}[S_3 = 1 \mid S_1 = 1]$.

b: Human calibration.

Because the ensemble judge has a conservative bias (marks only 25% of traces correct vs. 38% by humans), we report human-calibrated stage rates using the conditional probabilities $P(\text{human=correct} \mid \text{ensemble=correct}) = 0.76$ and $P(\text{human=correct} \mid \text{ensemble=wrong}) = 0.25$, estimated from the 100 human labels.

C. REJECTION–RECOVERY DECOMPOSITION

We decompose model robustness into two orthogonal capabilities:

- **Rejection rate** $= 1 - p_{S_1}$: the fraction of injection attempts the model rejects at the parameter stage.

- **Recovery rate** $= 1 - r_{2,3}$: among traces where the injection executed visibly ($S_2{=}1$), the fraction where the model still produced a correct final answer.

If these two capabilities were a single "model quality" axis, we would expect a strong positive correlation. We test independence using Spearman rank correlation across the nine models.

D. RUNTIME INTERCEPTOR

The Interceptor runs in parallel with the agent and consists of three layers:

- **L1 (schema validation)**: counts parameter validation errors and tool error responses; abstains if $\geq \tau_{L1}$ flags accumulate.
- **L2 (thought-keyword monitor)**: scans the agent's chain-of-thought for uncertainty keywords (*error, invalid, unknown, missing, incorrect, wrong, cannot*); triggers if $\geq \tau_{L2}$ hits occur.
- **L3 (output consistency)**: checks whether the final answer contains numerical claims unsupported by any tool observation.

Thresholds were selected via a 12-configuration sweep on a 40-row validation split ($\tau_{L1}{\in}\{1,2,3\}$, $\tau_{L2}{\in}\{2,3\}$, $L3{\in}\{\text{on,off}\}$). The best configuration ($\tau_{L1}{=}1$, $\tau_{L2}{=}2$, $L3{=}$on) achieved $F_1{=}0.842$ on the validation split. This F_1 is a sweep-internal optimum, not a held-out estimate.

E. STATISTICAL PROTOCOL

All confidence intervals are 95% bootstrap CIs (1,000 iterations, seed 42). Effect sizes for two-proportion comparisons use Cohen's h. Power is computed from the achieved sample sizes using the normal approximation. The interceptor experiment uses a concurrent no-interceptor control: both arms run the same 600 tasks per model at the same time, differing only in whether the Interceptor is active.

VI. RESULTS

A. JUDGE RELIABILITY

Table **??** reports Cohen's κ for each judge against the 100 human labels.

The heuristic substring match achieves $\kappa{=}0.049$ vs. human labels—statistically indistinguishable from chance. The three individual LLM judges achieve $\kappa{=}0.30$–0.44 vs. human, and the ensemble majority reaches $\kappa{=}0.432$ (moderate agreement per Landis–Koch [**?**]; Cohen's κ [**?**]). The ensemble is $8\times$ closer to human annotation than the heuristic, confirming that substring-based evaluation is unreliable for agent outputs.

a: Conservative bias.

Table **??** shows the ensemble-vs-human confusion matrix on 100 labels. The ensemble marks 25% of traces correct vs. 38% by humans. The dominant disagreement pattern is the ensemble rejecting answers that humans accept (19 out of 25 disagreements). This conservative bias means ensemble-judged correctness rates *underestimate* true correctness by approximately 13 percentage points.

TABLE 2. Judge reliability measured by Cohen's κ agreement. "κ vs. LLM majority" is computed on $N = 960$ P2 sessions; "κ vs. Human" is computed on $n = 100$ stratified-sampled sessions labeled by a domain expert. 95% CIs from 1,000-iteration bootstrap. Interpretation follows [?].

Judge	κ_{LLM}	κ_{Human}	95% CI	Interpretation
Heuristic	0.143	0.049	[-0.122, 0.221]	Slight
GPT-4o	0.792	0.336	[0.141, 0.533]	Fair
GPT-4o-mini	0.770	0.567	[0.391, 0.736]	Moderate
Gemini-2.5-Flash	0.821	0.379	[0.194, 0.546]	Fair
Ensemble (majority)	—	0.432	[0.239, 0.622]	Moderate

TABLE 3. Confusion matrix of the ensemble judge versus human expert labels ($n = 100$). Cells show count and row-percentage. Cohen's κ is reported in Table ??.

		Human label		
		Correct	Incorrect	Total
Ensemble	Correct	19 (19.0%)	6 (6.0%)	25
	Incorrect	19 (19.0%)	56 (56.0%)	75
	Total	38	62	100

b: Implications for reported numbers.

Throughout the remainder of this paper, we report both ensemble-judged and human-calibrated numbers. The ensemble provides the primary automated estimate; the human-calibrated version adjusts for the measured conservative bias.

We note that GPT-4o-mini individually achieves higher κ against humans (0.567) than the ensemble (0.432) on this 100-sample validation subset. The ensemble was retained because (i) a single judge may reflect idiosyncratic biases that a majority vote partially corrects, and (ii) the difference on $n{=}100$ is within overlapping confidence intervals (ensemble $[0.239, 0.622]$ vs. GPT-4o-mini $[0.391, 0.736]$). Larger human-labeled studies should re-examine this choice.

B. PROPAGATION RATES UNDER VALIDATED EVALUATION

Table **??** reports per-model stage-reach probabilities and hop-conditional rates on the 1,350 P2 semantic-wrong traces, judged by the validated ensemble.

a: Key findings.

Under ensemble judging, the overall hop-conditional rate from tool execution to wrong answer is $r_{2,3} = 0.78$ (123 of 158 traces with $S_2{=}1$ produced wrong answers). Human-calibrated, this drops to $r_{2,3} \approx 0.62$—a 16 percentage-point reduction attributable to the ensemble's conservative bias. Per-model $r_{2,3}$ ranges from 0.43 (DeepSeek-V3, $n_{S_2}{=}14$) to 0.97 (GPT-4.1-nano, $n_{S_2}{=}35$). The human-calibrated range is 0.46–0.73.

The propagation rate is substantial but highly variable across models. The per-model n_{S_2} values (2–45 for semantic-wrong) limit the precision of individual estimates. The injection-to-error rate $\mathbb{P}[S_3{=}1 \mid S_1{=}1]$ ranges from 0.57 to 0.90 (ensemble) and 0.53–0.70 (calibrated), providing a more robust marginal summary.

TABLE 4. Propagation-stage reach and hop rates under semantic-wrong injection. S_1: $\varepsilon \geq 1$; S_2: $\varepsilon \geq 2$; S_3: ensemble-judged incorrect. $r_{12} = P(S_2|S_1)$; $r_{23} = P(S_3|S_2)$; $P(S_3|S_1)$ is end-to-end propagation. 95% bootstrap CIs ($B = 1{,}000$).

Model	n	S_1%	CI	S_2%	CI	S_3%	CI	r_{12}	n_{S_2}	r_{23} CI	$P(S_3\|S_1)$	CI
DeepSeek-V3	200	52.0	[45.5, 59.0]	7.0	[3.5, 11.0]	62.5	[56.0, 69.0]	0.13	14	[21.4, 71.4]	0.57	[47.1, 66.4]
GPT-3.5-Turbo	200	37.5	[30.5, 44.0]	9.5	[5.5, 14.0]	67.5	[60.5, 73.5]	0.25	19	[52.6, 89.5]	0.76	[66.7, 85.3]
GPT-4.1-mini	200	50.5	[43.5, 57.0]	10.5	[6.5, 15.0]	65.5	[58.5, 71.5]	0.21	21	[47.6, 85.7]	0.66	[57.4, 76.2]
GPT-4.1-nano	200	62.5	[56.0, 69.5]	17.5	[12.0, 22.5]	84.5	[79.0, 89.5]	0.28	35	[91.4, 100.0]	0.83	[76.8, 88.8]
GPT-4o	50	74.0	[62.0, 86.0]	22.0	[10.0, 34.0]	74.0	[62.0, 86.0]	0.30	11	[54.5, 100.0]	0.68	[51.4, 81.1]
GPT-4o-mini	200	40.0	[33.0, 46.5]	22.5	[17.0, 28.0]	71.0	[64.5, 77.0]	0.56	45	[73.3, 95.6]	0.84	[76.2, 91.2]
Gemini-2.0-Flash	200	5.0	[2.5, 8.5]	1.0	[0.0, 2.5]	67.0	[60.5, 73.0]	0.20	2	[0.0, 100.0]	0.90	[70.0, 100.0]
Gemini-2.5-Flash	50	50.0	[36.0, 64.0]	6.0	[0.0, 14.0]	76.0	[64.0, 86.0]	0.12	3	[0.0, 100.0]	0.84	[68.0, 96.0]
Qwen2.5-14B	200	42.5	[36.0, 49.0]	9.5	[6.0, 14.0]	62.5	[55.5, 69.5]	0.22	19	[73.7, 100.0]	0.65	[55.3, 75.3]
Qwen2.5-3B	200	49.5	[42.5, 56.5]	14.0	[9.5, 19.0]	82.5	[77.0, 88.0]	0.28	28	[71.4, 96.4]	0.74	[65.6, 82.8]
Qwen2.5-7B	200	73.0	[66.5, 78.5]	28.0	[22.0, 34.5]	71.5	[65.5, 78.0]	0.38	56	[67.9, 91.1]	0.63	[55.5, 69.9]
hermes3$_{l}$*lama8b*	200	27.0	[21.0, 33.0]	5.0	[2.0, 8.5]	62.0	[55.5, 69.0]	0.19	10	[50.0, 100.0]	0.67	[53.7, 79.6]
o3-mini	50	74.0	[62.0, 86.0]	16.0	[6.0, 26.0]	70.0	[58.0, 82.0]	0.22	8	[25.0, 87.5]	0.76	[62.2, 89.2]

b: Baseline vs. P2 correctness.

Table **??** compares baseline and P2 correctness, both judged by the same 3-LLM ensemble. Because P2 traces include rows where the injection did not apply (eps=0), the unconditional P2 correctness is inflated by effectively clean traces. We therefore report P2 correctness conditioned on injection applied (eps≥1). Under this conditioning, P2 correctness is lower than baseline for five of seven models (e.g., GPT-4o-mini: 24.4% baseline vs. 16.2% P2|inj; o3-mini: 36.0% vs. 24.3%)—the expected direction for an injection protocol that makes tasks harder. DeepSeek-V3 is an exception (+15.9 pp), possibly because the injection-applied subset selects for structurally different task pairings; we flag this as an anomaly rather than a general trend.

Figure **??** visualizes the per-model stage-reach rates.

C. REJECTION AND RECOVERY ARE INDEPENDENT CAPABILITIES

Table **??** and Figure **??** present the rejection–recovery decomposition across the thirteen models (nine proprietary and four open-weight models; Section **??**).

Spearman rank correlation between rejection rate and recovery rate is ρ=−0.007 (p=0.983), indicating no relationship; models occupy a two-dimensional plane rather than a single quality axis. For example, GPT-4.1-nano shows moderate rejection (37.5%) but near-zero recovery (2.9%)—once an injection passes the schema gate, the model almost never recovers—whereas GPT-4o-mini pairs moderate rejection (60%) with low recovery (15.6%).

a: What "rejection" does and does not mean.

The rejection rate $1-p_{S_1}$ counts every trace in which the injection did *not* apply. This has two distinct causes that the raw rate conflates: (i) *genuine parameter rejection*—the model emitted a tool call but supplied a corrected or refused parameter; and (ii) *absent or fabricated tool use*—the model never emitted a parseable tool call at all, so there was no parameter to inject. Section **??** shows that these causes differ enormously across models: models with high tool-call rates (e.g. GPT-4o, GPT-4.1-nano, the Qwen2.5 models at 61–99%) express genuine rejection, whereas Gemini-2.0-Flash emits a real tool call in only 5% of traces and *fabricates* tool use in 42.5%. Its apparent 95% "rejection" is therefore dominated by tool-call avoidance, not parameter filtering. We report the tool-call rate alongside the rejection rate so the two mechanisms are not conflated. The practical implication stands: schema validation (rejection) and reasoning through corrupted observations (recovery) are independent, and mitigation should target them separately—but a third axis, whether the model reliably calls tools at all, must be measured first.

D. RUNTIME INTERCEPTOR

Table **??** reports the concurrent-control interceptor experiment.

a: GPT-4o-mini.

The interceptor reduces hallucination from 55.8% (control) to 32.8% (interceptor), a −23.0 percentage-point reduction. The interceptor saved 163 traces that would have been wrong and broke 25 that would have been correct. Analysis of the 25 broken traces revealed zero interceptor abstentions and no domain clustering—the failures are attributable to LLM run-to-run variance, not systematic false positives from the interceptor.

b: Gemini-2.0-Flash.

The interceptor has no significant effect: 44.5% (control) vs. 43.2% (interceptor), Δ=−1.3 pp, well within noise. The interceptor's abstention rate is only 2.8%. This null result follows directly from Gemini-2.0-Flash's tool-use behavior (Section **??**): it emits a real, injectable tool call in only 5% of traces and instead fabricates tool use in 42.5%. The interceptor validates tool calls, so when almost no tool call occurs there is nothing for it to catch. (This corrects an earlier interpretation that attributed the null result to "aggressive parameter rejection"; the true mechanism is absent or fabricated tool calls, not parameter filtering.) GPT-4o-mini, by contrast, emits real tool calls in 40% of traces, giving the interceptor genuine targets—hence its −23.0 pp effect.

TABLE 5. Baseline vs. P2 (semantic-wrong injection) task correctness per model. Correctness is judged by the 3-LLM ensemble for P2 and by heuristic match for baseline. 95% CIs via 1,000-iteration nonparametric bootstrap.

Model	n_{BL}	BL%	95% CI	n_{P2}	P2%	95% CI	Heur.%
DeepSeek-V3	450	51.8	[46.7, 56.4]	200	37.5	[31.0, 44.0]	70.5
GPT-3.5-Turbo	450	50.4	[45.8, 54.9]	200	32.5	[26.5, 39.5]	67.5
GPT-4.1-mini	450	54.7	[50.2, 59.3]	200	34.5	[28.5, 41.5]	70.0
GPT-4.1-nano	450	45.3	[40.4, 50.0]	200	15.5	[10.5, 21.0]	67.5
GPT-4o	—	0.0	—	50	26.0	[14.0, 38.0]	62.0
GPT-4o-mini	450	40.0	[35.3, 44.4]	200	29.0	[23.0, 35.5]	49.0
Gemini-2.0-Flash	450	47.6	[42.7, 52.2]	200	33.0	[27.0, 39.5]	61.0
Gemini-2.5-Flash	—	0.0	—	50	24.0	[14.0, 36.0]	64.0
Qwen2.5-14B	450	55.6	[51.3, 60.0]	200	37.5	[30.5, 44.5]	70.5
Qwen2.5-3B	450	54.2	[49.6, 58.7]	200	17.5	[12.0, 23.0]	67.5
Qwen2.5-7B	450	49.1	[44.2, 53.8]	200	28.5	[22.0, 34.5]	57.0
hermes3$_l$*lama8b*	450	58.2	[53.8, 62.2]	200	38.0	[31.0, 44.5]	71.5
o3-mini	50	40.0	[28.0, 54.0]	50	30.0	[18.0, 42.0]	54.0

TABLE 6. Rejection rate ($1 - P(S_1)$) versus recovery rate ($1 - r_{23}$) per model. Spearman $\rho = 0.041$, $p = 0.893$ ($n = 13$ models), indicating no significant monotonic relationship between a model's propensity to reject injected errors and its ability to recover from propagated ones.

Model	Rejection%	Recovery%	n_{S_2}
DeepSeek-V3	48.0	57.1	14
GPT-3.5-Turbo	62.5	26.3	19
GPT-4.1-mini	49.5	33.3	21
GPT-4.1-nano	37.5	2.9	35
GPT-4o	26.0	18.2	11
GPT-4o-mini	60.0	15.6	45
Gemini-2.0-Flash	95.0	50.0	2
Gemini-2.5-Flash	50.0	33.3	3
Qwen2.5-14B	57.5	10.5	19
Qwen2.5-3B	50.5	14.3	28
Qwen2.5-7B	27.0	19.6	56
hermes3$_l$*lama8b*	73.0	20.0	10
o3-mini	26.0	37.5	8

c: Honest limitation.

The −23.0 pp delta is measured under heuristic judging applied identically to both arms. The delta is robust to shared judging bias (any systematic over- or under-counting affects both arms equally). However, the absolute hallucination rates in each arm (55.8% and 32.8%) have not been validated by human labeling. Validating these absolute rates would require additional human annotation of the interceptor runs, which we leave to future work.

E. TOOL-CALL ADHERENCE AND FABRICATED TOOL USE

A parameter-level injection can only apply when the agent actually emits a parseable tool call. We therefore measure, for each model, the fraction of P2 semantic-wrong traces that contain a tool call (the tool-call rate) and, among traces with no tool call, the fraction whose final answer nonetheless asserts a completed tool-derived result—a behavior we term *fabricated tool use* (e.g., "I have retrieved the weather. . ." or "I have scheduled the event. . ." with no tool ever invoked).

Table **??** shows that tool-call adherence varies by more than an order of magnitude. GPT-4o and GPT-4.1-nano call tools in 99–100% of traces, whereas Gemini-2.0-Flash does so in only 5% and fabricates tool use in 42.5% of all traces; GPT-4o-mini calls tools 40% of the time and fabricates 24%. This has two consequences. First, it explains the interceptor results (Section **??**): a runtime monitor that validates tool calls is effective only when the model emits them. Second, it cautions against reading a low stage-one rate as robustness—for Gemini-2.0-Flash the low p_{S_1} reflects fabricated or absent tool calls, not parameter rejection. Fabricated tool use is itself a hallucination mode that end-to-end correctness scores and substring judges do not surface.

a: Native function-calling versus prompted tool use.

The rates in Table **??** are measured under a prompted-ReAct harness applied identically to all models. To test whether a low rate is harness-specific, we re-measured the tool-call rate under vLLM's *native* function-calling interface (`tools=`, `tool_choice=auto`) for the open-weight models that support it. For Qwen2.5-7B the rate is essentially unchanged (99.5% prompted vs. 98.5% native), but for Qwen2.5-14B it rises substantially (61.0% to 93.5%). Thus, for a model that supports native tool-calling, a low prompted rate partly reflects the harness rather than an inability to use tools. This avenue is not universal, however: Mistral-7B-v0.2 and Phi-3.5-mini expose no native tool-calling interface in vLLM (the server cannot locate a tool-call token in the tokenizer), so

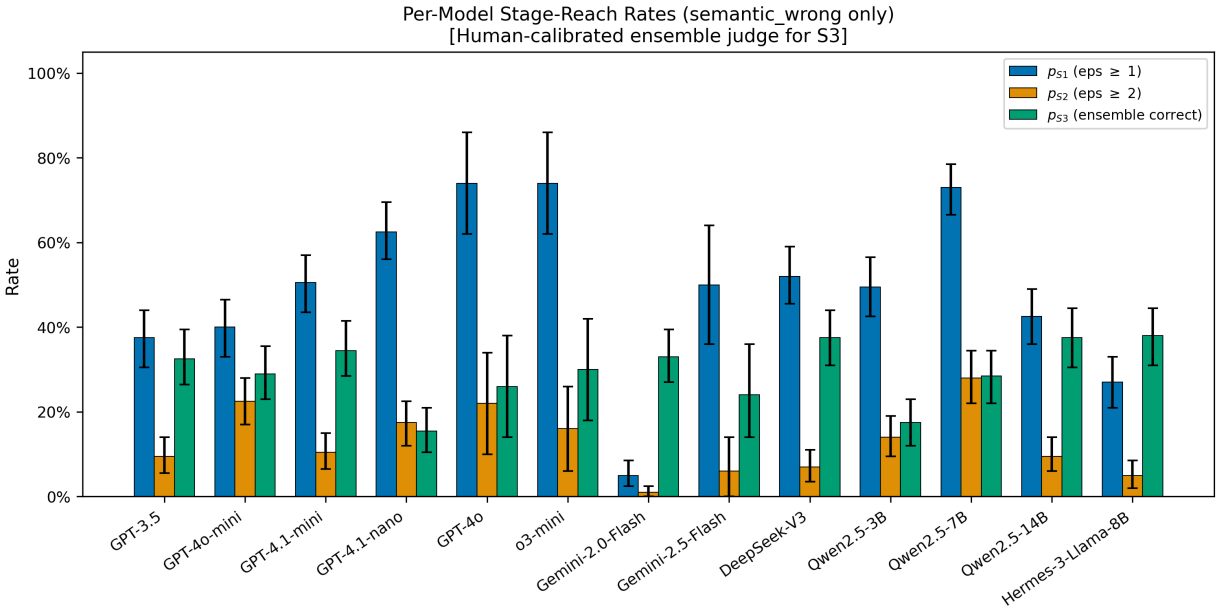


FIGURE 1. **Per-model stage-reach rates under P2 semantic-wrong injection (ensemble-judged). S_1: injection applied; S_2: visible execution effect; S_3: wrong final answer. Stage indicators are computed independently.**

TABLE 7. **Interceptor concurrent-control experiment. Control arm runs P2 tasks without the interceptor; interceptor arm uses the tuned three-layer filter. "Saved" = hallucinations correctly abstained; "Broken" = correct answers wrongly abstained. 95% CIs on ´ via 1,000-iteration bootstrap on per-session hallucination indicator.**

Model	Ctrl hall.%	Int. hall.%	Δ pp	95% CI	Abstain%	Saved	Broken
GPT-4o-mini	55.8	32.8	-23.0	[-28.5, -17.3]	33.5	133	68
Gemini-2.0-Flash	44.5	43.2	-1.3	[-7.3, 4.2]	2.8	16	1

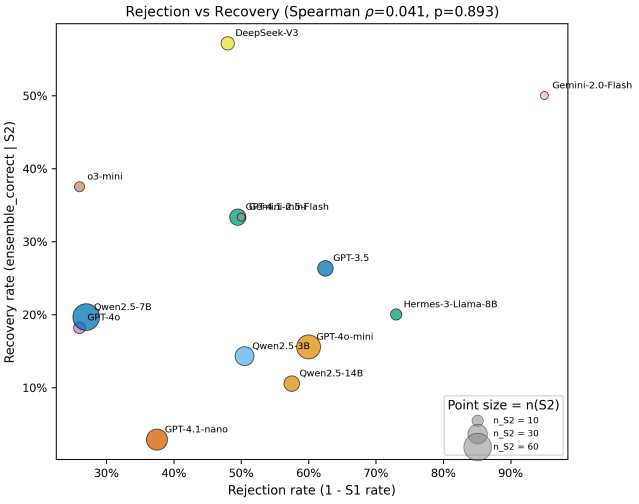


FIGURE 2. **Rejection rate ($1-p_{S_1}$) vs. recovery rate ($1-r_{2,3}$) for thirteen models. Point size proportional to n_{S_2}. Spearman ρ=0.041, p=0.893: axes are statistically independent.**

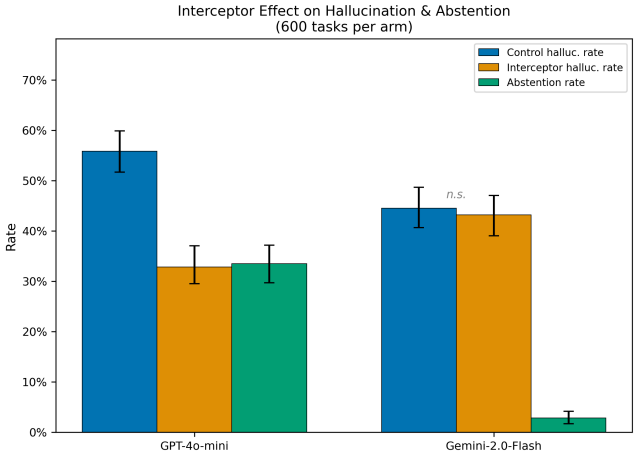


FIGURE 3. **Interceptor effect under concurrent n=600 control. GPT-4o-mini shows a significant 23 pp reduction; Gemini-2.0-Flash shows no significant effect. Error bars: 95% bootstrap CIs.**

their near-absent tool use is a genuine capability gap, not a harness artifact. We therefore treat the tool-call rate as harness-conditional, and note that fabricated tool use remains most consequential in the text-based (prompted) agent loops that are still common in deployment and in which models without native tool interfaces must operate.

TABLE 8. Tool-call rate and fabricated-tool-use rate on P2 semantic-wrong traces (%). Fabricated = no tool call emitted, yet the final answer asserts a completed tool-derived result.

Model	Tool-call rate	Fabricated
GPT-4o	100.0	0.0
GPT-4.1-nano	99.0	0.5
o3-mini	88.0	8.0
DeepSeek-V3	86.0	7.0
GPT-4.1-mini	85.0	12.0
Gemini-2.5-Flash	76.0	18.0
GPT-3.5-turbo	59.5	21.5
GPT-4o-mini	40.0	24.0
Gemini-2.0-Flash	5.0	42.5
Qwen2.5-7B	99.5	—
Qwen2.5-3B	70.0	—
Qwen2.5-14B	61.0	—
Hermes-3-Llama-3.1-8B	37.0	—
Mistral-7B-v0.2	21.5	—
Phi-3.5-mini	0.0	—

F. GENERALIZATION TO OPEN-WEIGHT MODELS

To test whether the propagation findings generalize beyond proprietary APIs, we evaluated four open-weight models spanning two families, served locally with vLLM under the identical prompted-ReAct harness: the Qwen2.5 series at three scales (3B, 7B, 14B) and Hermes-3-Llama-3.1-8B (a Llama-3.1 derivative). Their injection-to-error rates $\mathbb{P}[S_3{=}1 \mid S_1{=}1]$ are 0.74 (Qwen-3B), 0.63 (Qwen-7B), 0.65 (Qwen-14B), and 0.67 (Hermes-3-Llama-8B), with hop-conditional rates $r_{2,3}$ of 0.86, 0.80, 0.89, and 0.80 respectively—all within the proprietary-model ensemble range (0.57–0.90) and consistent with the human-calibrated proprietary estimate (≈ 0.62). Propagation is thus substantial across model scale and across two open families, confirming it is not an artifact of proprietary model APIs. (The Qwen models are competent tool users at 61–99.5%; Hermes-3-Llama emits tool calls in 37% of traces, enough to yield a stable estimate at $n_{S_2}{=}10$, and its low rate is the same tool-call-avoidance phenomenon discussed in Section **??**.)

We also attempted two other open families. Mistral-7B-Instruct-v0.2 and Phi-3.5-mini did not reliably follow the zero-shot ReAct tool-use protocol (tool-call rates 21.5% and 0%; Table **??**), emitting final answers without tool calls, so their propagation could not be measured. This is itself an evaluation-relevant observation: agentic tool-use competence varies widely among open instruct models of similar size, and a benchmark of tool-using agents must verify that a model actually uses tools before scoring its robustness.

G. CROSS-DOMAIN REPLICATION ON HELD-OUT RETAIL

To test generalization, we ran baseline and P2 experiments on 300 held-out retail tasks for three models. Baseline correctness is lower on retail (harder three-tool chains), but the model rank ordering is preserved on both correctness and mean EPS, consistent with the original-domain results.

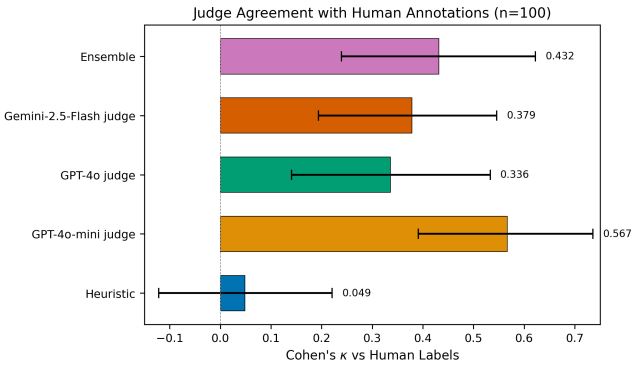


FIGURE 4. Cohen's κ vs. 100 human labels for each judge method. The heuristic is at chance level (κ=0.049); the ensemble reaches moderate agreement (κ=0.432). Error bars: 95% bootstrap CIs.

VII. DISCUSSION AND IMPLICATIONS

A. THEORETICAL IMPLICATIONS

a: Rejection and recovery as orthogonal axes.

The finding that rejection rate and recovery rate are statistically independent (Spearman $\rho{=}0.041$, $p{=}0.893$) suggests that agent robustness to tool-use errors is at least two-dimensional. A model's ability to reject bad parameters at the schema gate is architecturally distinct from its ability to reason through corrupted observations. This has implications for how we model agent reliability: a single "robustness score" is insufficient; instead, practitioners should evaluate both the input-filtering and the output-reasoning capabilities of an agent independently.

b: Why heuristic evaluation misranks models.

The heuristic over-counted correctness by 34 percentage points on P2 traces, enough to reverse the direction of the baseline-vs-P2 comparison. This occurs because injected errors produce paraphrased wrong answers that contain enough keywords to pass substring matching. Any benchmark that uses heuristic evaluation on agent outputs with high output variability risks similar misranking.

B. PRACTICAL IMPLICATIONS FOR AGENT DESIGNERS

a: Schema validators are high-leverage.

Gemini-2.0-Flash's 95% rejection rate demonstrates that strict parameter validation can prevent most injection-type errors from reaching the tool-execution stage. Designers should invest in comprehensive tool schemas with explicit type, range, and enum constraints.

b: Pre-execution semantic checks.

The Interceptor's L1 (schema) and L2 (keyword) layers provided the bulk of its discriminative power. For models with moderate rejection rates (e.g., GPT-4o-mini at 60%), a runtime interceptor can halve the hallucination rate at minimal cost ($0.15 for 1,200 runs).

c: Interceptors are model-dependent.

The null result on Gemini-2.0-Flash shows that interceptors are not universally effective. When a model already rejects most errors at the parameter stage, there is nothing left for the interceptor to catch. Deployment decisions should be informed by model-specific rejection rates.

C. IMPLICATIONS FOR IR AND IS EVALUATION

a: Substring heuristics should be deprecated.

Our κ=0.049 result is the lowest heuristic-vs-human agreement we are aware of in any agent or QA evaluation study. The IR community has already raised concerns about automated evaluation reliability [**?**], [**?**]. Our result provides concrete evidence that substring heuristics are worse than coin flips in the agent-evaluation context.

b: LLM-ensemble judges need validation.

The ensemble's κ=0.432 is a significant improvement but falls short of the substantial-agreement threshold (κ>0.60). The ensemble has a measured conservative bias, underreporting correctness by $\sim$13 pp relative to humans. We recommend that any study using LLM judges report: (a) agreement with at least 50 human labels, (b) the direction of bias (conservative vs. permissive), and (c) calibrated estimates alongside raw judge outputs.

c: Methodology transfers to RAG and QA evaluation.

The three-judge ensemble protocol, the human-calibration procedure, and the independence-testing framework for evaluation axes all apply to RAG pipeline evaluation, open-domain QA, and any setting where output format variability makes heuristic matching unreliable.

D. DIFFERENTIATION FROM EXISTING WORK

Table **??** positions AgentProp-Bench relative to four closely related benchmarks.

TABLE 9. Comparison with related benchmarks.

Feature	τ-bench	AgentBench	AgentHallu	AgentProp-Bench
Stage-level metrics	×	×	✓	✓
Controlled injection	×	×	×	✓
Human-validated judge	×	×	×	✓
Runtime mitigation	×	×	×	✓
Concurrent control	×	×	×	✓
# Models evaluated	6	25	14	13
# Traces	1,280	7,000+	693	14,750

VIII. LIMITATIONS

1) **Ensemble conservative bias.** The ensemble judge's $P(\text{human=correct} \mid \text{ensemble=wrong}) = 0.25$ means one in four ensemble "wrong" verdicts is a false negative. All ensemble-judged rates should be interpreted as lower bounds on true correctness.
2) **Human-label sample size.** With n=100 labels, the calibration estimates have wide confidence intervals. The human-vs-ensemble κ=0.432 has a 95% CI of approximately $[0.28, 0.58]$ by bootstrap. This is moderate, not definitive, validation. The primary annotator was the first author; a second annotator independently labeled all 100 traces with almost-perfect agreement (Cohen's κ=0.835), mitigating single-annotator bias, though both annotators share a similar background. Future work should use a larger non-author annotator pool to eliminate any potential annotator bias.
3) **Interceptor absolute rates.** The interceptor delta (-23.0 pp) is robust to shared heuristic judging bias, but the absolute hallucination rates (55.8% and 32.8%) are unvalidated by human annotation. Future work should apply human labeling to the interceptor arms.
4) **Model selection.** We evaluated nine proprietary models and four open-weight models across two families (Qwen2.5 3B/7B/14B and Hermes-3-Llama-3.1-8B). Injection operates at the prompted tool-call layer, which is model-agnostic, so the same harness applies to locally-served open-weight models without modification. Two further open families (Mistral-7B-v0.2, Phi-3.5-mini) were evaluated but did not reliably emit tool calls (Section **??**), precluding propagation measurement. We did not evaluate very large open-weight models ($\geq$70B) that exceed a single 24 GB GPU, nor Claude models (to avoid potential evaluation conflicts); both are natural extensions.
5) **Simulated tools.** All tool calls execute against deterministic simulators, not live APIs. Live API variability (rate limits, format changes, partial failures) may introduce additional propagation pathways not captured here.
6) **Single-parameter injection.** We inject one parameter per trace. Multi-parameter and multi-turn injection (P3 memory accumulation, P4 cross-agent propagation) are deferred to future work.
7) **Scope.** This paper covers only the P2 parameter-injection protocol. P3 (memory-accumulation propagation across turns) and P4 (cross-agent pipeline propagation) share the EPS machinery but require different task formats and are deferred.
8) **Small per-model n_{S_2} samples.** The stage-2 subset sizes n_{S_2} range from 2 (Gemini-2.0-Flash) to 45 (GPT-4o-mini). Per-model $r_{2,3}$ estimates for models with $n_{S_2} < 10$ have wide confidence intervals (e.g., CI $[0, 1]$ for n_{S_2}=2) and should be interpreted as directional only. The marginal injection-to-error rate $\mathbb{P}[S_3 \mid S_1]$, computed on the larger S_1=1 subset, provides a more stable summary.
9) **Power of the independence test.** The Spearman correlation between rejection and recovery rates is computed across n=12 models. With this sample size, only strong monotonic relationships ($|\rho| \gtrsim 0.6$) would be detectable at conventional significance levels. Our finding

(ρ=0.041, p=0.893) is consistent with genuine independence but also with a moderate underlying relationship the test cannot detect. We report it as *no evidence of correlation* rather than *proof of independence*.

IX. CONCLUSION

We introduced AgentProp-Bench, a benchmark for evaluating tool-using LLM agents with validated judging, stage-level propagation measurement, and a runtime mitigation evaluated under concurrent control.

a: RO1 (Judge reliability).
Substring-based evaluation of agent outputs agrees with human annotation at κ=0.049 (chance-level). A three-LLM ensemble reaches κ=0.432 (moderate) with a conservative bias. We recommend that all agent benchmarks validate their evaluation methodology against human labels before reporting comparative claims.

b: RO2 (Propagation).
Under validated evaluation, a parameter-level injection propagates to a wrong final answer with human-calibrated probability $\approx$0.62 (range 0.46–0.73 across models); four open-weight models across two families (Qwen2.5 3B/7B/14B and Hermes-3-Llama-3.1-8B) reproduce this (injection-to-error 0.63–0.74), confirming it is not an artifact of proprietary APIs. Rejection rate and recovery rate are independent capabilities (Spearman ρ=0.041, p=0.893), indicating that model robustness to tool-use errors is at least two-dimensional. We also find that models fabricate tool use—asserting tool results they never obtained—so a low injection rate can reflect tool-call avoidance rather than robustness.

c: RO3 (Mitigation).
A lightweight runtime interceptor reduces GPT-4o-mini hallucination by 23 percentage points under a concurrent n=600 control. The effect is model-dependent: Gemini-2.0-Flash shows no effect because it rarely emits an injectable tool call (5%), leaving a tool-call-validating interceptor with almost nothing to catch.

All code, data, 14,750 agent traces, and 100 human labels are released at https://github.com/bhaskargurram-ai/agenthallu-bench to support reproducible agent evaluation research.

DECLARATIONS

Competing Interests: None declared. Zasti Inc. is the author's employer; no external funder influenced the research.

Funding: Self-funded with employer support.

Data and Code: https://github.com/bhaskargurram-ai/agenthallu-bench.

ACKNOWLEDGMENT

The author thanks Yoshitha Challagulla for independently annotating a subset of traces used in the inter-annotator agreement analysis.

BHASKAR GURRAM received the M.S. degree in computer science. He is currently a Machine Learning Scientist with Zasti Inc., Ashburn, VA, USA. His research interests include the evaluation and reliability of tool-using large language model agents, retrieval and information systems, and runtime monitoring of autonomous agents.